



Université Cheikh Anta Diop de Dakar

Systèmes distribués et services web

Rapport de travaux pratiques

Projet i-fall / ChatUser

Objet du document

Présenter de manière claire, structurée et détaillée les réalisations autour de l'application ChatUser : TCP sockets, RMI, XML-RPC, SOAP, REST, comparaison des architectures, guide d'exécution, puis annexes Ant et FOP.

Étudiants	Salif Biaye Ndeye Astou Diagouraga Mouhamadou Tidiane Seck Sountou Sakho
Enseignant	Pr. Ibrahima Fall
Nature du livrable	Rapport Word complet, sans LaTeX
Technologies étudiées	TCP sockets, Java RMI, XML-RPC, SOAP, REST, Ant, Apache FOP
Code source	https://github.com/salifbiaye/projet-tp-0
Date	Mai 2026

Version académique structurée avec schémas explicatifs et extraits de code commentés

Code source : <https://github.com/salifbiaye/projet-tp-0>

Sommaire

Plan manuel cliquable du document

Partie	Accès
Introduction générale aux systèmes distribués	Lien
Application ChatUser avec sockets TCP	Lien
Application ChatUser avec Java RMI	Lien
Application ChatUser avec XML-RPC	Lien
Application ChatUser avec SOAP	Lien
Application ChatUser avec REST	Lien
Comparaison générale des architectures	Lien
Guide d'exécution, validation et captures	Lien
Annexes : Projet Ant, Apache FOP et références	Lien

Table des figures

Figure	Titre
Figure 1.1	Architecture commune du projet ChatUser
Figure 2.1	Communication TCP Socket bas niveau
Figure 3.1	Architecture Java RMI avec registre et callbacks
Figure 4.1	Communication XML-RPC avec appel HTTP et polling
Figure 5.1	Traitement SOAP avec enveloppe XML et WSDL
Figure 6.1	Architecture REST avec endpoints HTTP et JSON
Figure 7.1	Comparaison des modèles de communication distribuée
Figure 8.1	Chaîne de validation commune des applications ChatUser
Figure A.1	Pipeline Apache Ant du projet
Figure A.2	Pipeline Apache FOP de génération PDF

Chapitre 1 : Introduction générale aux systèmes distribués

Comprendre le contexte avant les implémentations

1.1 Contexte

Le projet i-fall porte sur une application de discussion nommée ChatUser. Le besoin fonctionnel reste volontairement simple : plusieurs utilisateurs doivent pouvoir rejoindre une salle de discussion, envoyer des messages et recevoir les messages des autres participants. Cette simplicité est utile pédagogiquement, car elle permet de comparer plusieurs technologies distribuées sans changer le métier de l'application.

Dans un système distribué, les composants ne s'exécutent pas forcément dans la même machine virtuelle, ni même sur la même machine physique. Il faut donc définir un moyen de communication entre client et serveur : socket réseau, appel d'objet distant, appel de procédure distante, échange HTTP, message XML ou message JSON. Le projet illustre ces approches à travers des versions Java différentes.

Principe directeur

Le rapport ne se contente pas de décrire les fichiers : il explique la logique de chaque architecture, les choix de communication et la manière de tester le comportement attendu.

1.2 Objectif fonctionnel commun

Toutes les variantes reposent sur le même scénario : un serveur maintient l'état de la salle, un client saisit un pseudo, le serveur accepte ou refuse l'inscription, puis les messages sont diffusés ou récupérés selon la technologie employée. La comparaison est donc fiable, parce que le besoin reste stable alors que le mécanisme réseau change.

- Inscrire un utilisateur avec un pseudo unique.
- Envoyer un message depuis un client vers la salle commune.
- Mettre les nouveaux messages à disposition des autres clients.
- Gérer la sortie d'un utilisateur et notifier les autres participants.
- Compiler, exécuter et archiver les projets avec Apache Ant.

La figure 1.1 présente architecture commune du projet chatuser.

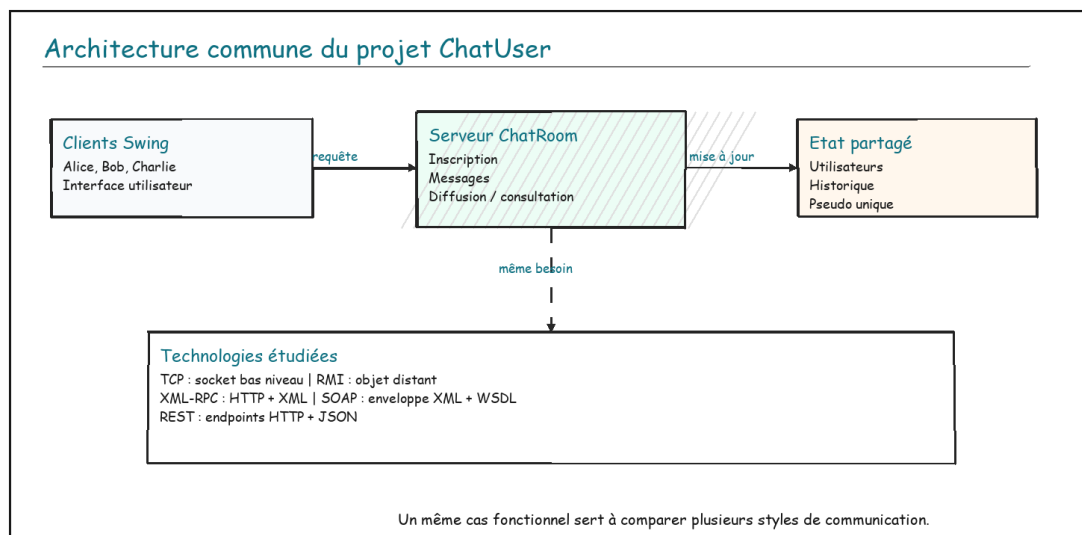


Figure 1.1 : Architecture commune du projet ChatUser

1.3 Périmètre retenu

Le rapport commence volontairement par TCP sockets, car c'est l'approche la plus basse : elle montre la connexion IP/port et l'échange de données brutes avant les abstractions. Les chapitres suivants montent progressivement en niveau : RMI masque le réseau derrière l'objet Java, XML-RPC et SOAP passent par HTTP/XML, puis REST exploite directement les ressources HTTP et JSON. Les dossiers Ant et FOP sont placés en annexes techniques : ils complètent le livrable sans alourdir les chapitres principaux.

Partie	Dossier source	Rôle dans le rapport
TCP sockets	TP0-TCP	Chapitre principal : communication IP/port et protocole ligne par ligne.
Java RMI	TP0	Chapitre principal : objets distants et callbacks.

Partie	Dossier source	Rôle dans le rapport
XML-RPC	TP0-XMLRPC	Chapitre principal : RPC simple sur HTTP/XML.
SOAP	TP0-SOAP	Chapitre principal : enveloppe XML et contrat WSDL.
REST	TP0-REST	Chapitre principal : endpoints HTTP et JSON.
Apache Ant	ProjetAnt et build.xml	Annexe : automatisation compilation, documentation et archive.
Apache FOP	fop et projet	Annexe : transformation XML/XSL-FO vers PDF.

Chapitre 2 : Application ChatUser avec sockets TCP

Communication IP, port et protocole ligne par ligne

2.1 Contexte du travail pratique

La version TCP est placée en premier parce qu'elle représente le niveau le plus bas étudié dans le projet. Avant de parler d'objet distant, de XML, de SOAP ou de REST, il faut comprendre qu'au fond une application distribuée doit faire communiquer deux machines à travers une adresse, un port et un flux de données.

Dans cette variante, le serveur ChatUser ne publie pas encore de service web ni d'objet distant. Il écoute simplement sur un port avec `ServerSocket`, accepte les connexions entrantes et échange des lignes de texte avec les clients connectés.

La figure 2.1 présente communication tcp socket bas niveau.

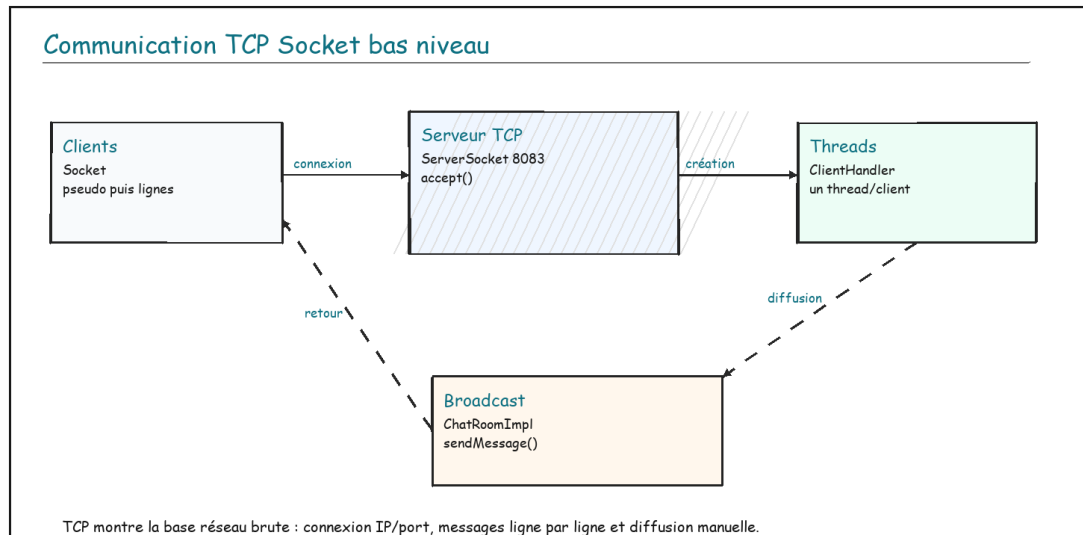


Figure 2.1 : Communication TCP Socket bas niveau

2.2 Fonctionnement

Le serveur ouvre un `ServerSocket` sur le port 8083. À chaque connexion entrante, il crée un `ClientHandler` exécuté dans un thread séparé. Le client envoie d'abord son pseudo, puis chaque ligne suivante est interprétée comme un message de chat.

Étape	Message ou action	Traitement côté serveur
1	Ouverture d'une socket vers localhost:8083	Le `ServerSocket` accepte la connexion entrante.
2	Envoi du pseudo sur la première ligne	Le serveur valide le pseudo et répond `OK` ou `ERROR`.
3	Envoi d'une ligne de texte	La ligne est considérée comme un message de chat.
4	Diffusion	`ChatRoomImpl` parcourt les clients connectés et appelle `sendMessage`.
5	Déconnexion	Le client est retiré de la liste et les autres utilisateurs sont notifiés.

Extrait ciblé : acceptation des connexions TCP et création d'un thread par client.

```

try (ServerSocket serverSocket = new ServerSocket(8083)) {
    while (true) {
        Socket socket = serverSocket.accept();
        ClientHandler handler = new ClientHandler(socket);
        new Thread(handler).start();
    }
}

```

Cette approche est plus manuelle : il faut gérer le protocole ligne par ligne, les connexions, les déconnexions et la diffusion. Elle est utile pour apprendre, mais elle demande plus de discipline qu'un service HTTP ou qu'un appel distant déjà structuré.

2.3 Analyse et limites

La variante TCP rend visibles les détails que les autres technologies cachent : ouverture de socket, flux d'entrée/sortie, boucle d'acceptation et concurrence. Cette transparence est intéressante pour apprendre le réseau, mais elle impose de définir soi-même un protocole applicatif. Ici, le protocole est volontairement simple : une ligne pour le pseudo, puis une ligne par message.

Les limites apparaissent vite dans une application réelle : pas de format structuré comme JSON ou XML, pas de contrat de service, pas de gestion avancée des erreurs et un risque de complexité dès qu'il faut ajouter des commandes plus riches. Cette observation justifie le passage aux technologies suivantes : RMI, XML-RPC, SOAP et REST ajoutent chacune une abstraction au-dessus de cette base réseau.

Transition vers RMI

TCP donne le contrôle total, mais oblige le développeur à tout gérer. RMI arrive ensuite pour masquer une partie de cette complexité dans le monde Java : le client ne manipule plus seulement des lignes de texte, il appelle une méthode distante.

Chapitre 3 : Application ChatUser avec Java RMI

Objet distant, registre RMI et callbacks

3.1 Contexte du travail pratique

La version RMI du projet montre comment une application Java peut appeler les méthodes d'un objet situé dans une autre JVM. Le serveur publie un objet distant représentant la salle de discussion, tandis que chaque client expose aussi un objet distant permettant de recevoir les messages par callback.

3.2 Objectif du travail pratique

L'objectif est de réaliser une salle de discussion distribuée en conservant une programmation orientée objet. Le travail attendu consiste à définir les interfaces distantes, implémenter la salle commune, publier l'objet serveur dans le registre RMI et vérifier que plusieurs clients reçoivent bien les messages diffusés.

La réponse proposée repose donc sur deux contrats : `ChatRoom` pour les opérations de la salle et `ChatUser` pour le retour vers les clients. Cette double interface permet d'obtenir une communication bidirectionnelle : le client appelle le serveur et le serveur rappelle le client.

3.3 Notions utilisées

Java RMI repose sur trois idées : une interface distante, une implémentation exportée et un registre qui permet au client de retrouver l'objet distant par son nom. Dans ce projet, `ChatRoom` est l'interface distante côté serveur, tandis que `ChatUser` permet au serveur de rappeler les clients connectés.

Point important

RMI garde une écriture très proche de l'objet local : le client appelle `postMessage`, mais l'exécution réelle se produit côté serveur. Le réseau est masqué par l'infrastructure RMI.

La figure 3.1 présente architecture java rmi avec registre et callbacks.

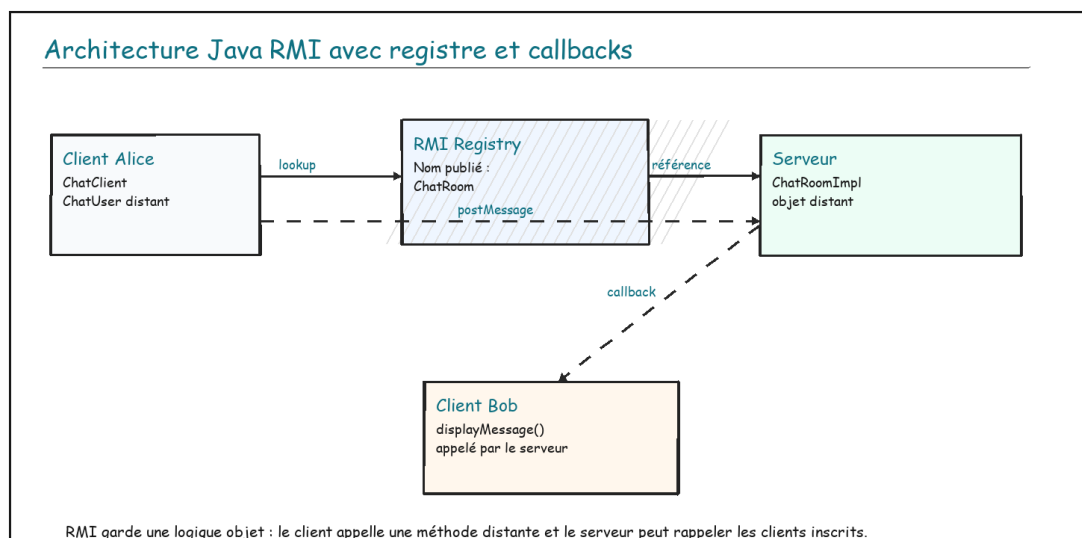


Figure 3.1 : Architecture Java RMI avec registre et callbacks

3.4 Analyse du code

L'interface `ChatRoom` définit les opérations accessibles à distance. Elle hérite de `Remote`, et chaque méthode déclare `RemoteException`, ce qui rappelle qu'un appel distant peut échouer pour une raison réseau, une indisponibilité du serveur ou une déconnexion du client.

Extrait ciblé : interface distante `ChatRoom` dans la version RMI.

```

public interface ChatRoom extends Remote {
    void subscribe(ChatUser user, String pseudo) throws RemoteException;
    void unsubscribe(String pseudo) throws RemoteException;
    void postMessage(String pseudo, String message) throws RemoteException;
}
  
```

L'implémentation `ChatRoomImpl` conserve une table de correspondance entre pseudo et objet `ChatUser`. Lorsqu'un message est reçu, la méthode privée `broadcastMessage` parcourt les utilisateurs et appelle

`displayMessage` sur chacun. Ce choix correspond à une diffusion active : le serveur pousse le message vers les clients.

Élément	Rôle	Justification
ChatRoom	Contrat distant du serveur	Expose les actions de la salle de discussion.
ChatUser	Contrat distant du client	Autorise le callback `displayMessage`.
ChatRoomImpl	État de la salle	Garde la liste des utilisateurs et diffuse les messages.
ChatServer	Publication RMI	Crée le registre et associe le nom `ChatRoom` à l'objet.
ChatClient	Interface utilisateur	Récupère la référence distante et appelle les méthodes.

3.5 Commandes d'exécution

Le projet possède un fichier `build.xml`. Les commandes Ant évitent de compiler manuellement chaque paquet Java.

Commandes de compilation et lancement de la version RMI.

```
cd C:\Users\DELL\Downloads\i-fall\TP0
ant compile
ant run-server

# dans un autre terminal
ant run-client -Dpseudo=Alice
```

3.6 Résultat attendu et synthèse

Le serveur doit afficher les arrivées, les départs et les messages. Deux clients lancés en parallèle doivent voir les mêmes événements de conversation. La version RMI est très adaptée à un environnement Java homogène, mais elle est moins naturelle si des clients non Java doivent communiquer avec le service.

La principale limite est donc l'interopérabilité. En revanche, pour apprendre les objets distribués, RMI est très parlant : la séparation entre interface, implémentation, registre et client montre proprement les responsabilités d'une application distribuée Java.

Chapitre 4 : Application ChatUser avec XML-RPC

Appels de procédures distantes sur HTTP et XML

4.1 Contexte du travail pratique

XML-RPC propose une approche différente de RMI. Le client n'obtient pas une référence vers un objet Java distant ; il envoie une requête HTTP contenant le nom d'une méthode et ses paramètres encodés en XML. Le serveur reçoit la requête, identifie la méthode appelée et renvoie une réponse XML.

4.2 Objectif du travail pratique

L'objectif est de réimplémenter le même besoin ChatUser en remplaçant l'appel objet distant par un appel de procédure distante. Le serveur doit publier des méthodes accessibles par nom, tandis que le client doit construire des appels XML-RPC et interpréter les réponses.

La réponse proposée choisit une liste centralisée de messages et un index de lecture. Cette solution évite d'avoir un objet client distant et s'adapte mieux à un protocole requête/réponse : le client revient périodiquement chercher ce qu'il n'a pas encore lu.

4.3 Fonctionnement retenu dans ChatUser

Dans cette version, le serveur expose un gestionnaire nommé `ChatRoom`. Les opérations principales sont `subscribe`, `unsubscribe`, `postMessage` et `getMessages`. Comme le serveur XML-RPC ne rappelle pas directement les clients, la réception repose sur le polling : le client demande périodiquement les nouveaux messages depuis un index connu.

La figure 4.1 présente communication xml-rpc avec appel http et polling.

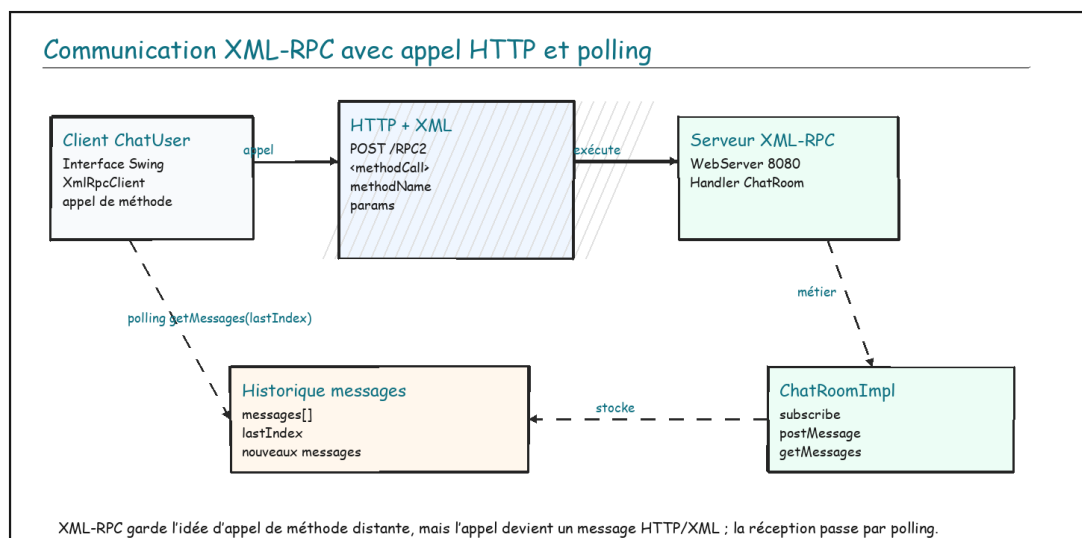


Figure 4.1 : Communication XML-RPC avec appel HTTP et polling

Différence avec RMI

RMI autorise un callback direct vers l'objet client. XML-RPC reste sur un échange requête/réponse : pour recevoir les messages, le client doit revenir demander ce qui a changé.

4.4 Analyse du code

Le serveur s'appuie sur `WebServer` de la bibliothèque Apache XML-RPC. Le mapping associe le nom logique `ChatRoom` à la classe `ChatRoomImpl`. Côté métier, la classe conserve une liste de messages et retourne uniquement les messages ajoutés depuis `lastIndex`.

Extrait ciblé : publication du gestionnaire XML-RPC côté serveur.

```

WebServer webServer = new WebServer(8080);
XmlRpcServer xmlRpcServer = webServer.getXmlRpcServer();

PropertyHandlerMapping phm = new PropertyHandlerMapping();
phm.addHandler("ChatRoom", ChatRoomImpl.class);
xmlRpcServer.setHandlerMapping(phm);

webServer.start();
  
```

Le choix de `lastIndex` évite de renvoyer l'historique complet à chaque appel. Le client garde la position du dernier message reçu, puis le serveur calcule la sous-liste à retourner. Ce mécanisme est simple, lisible et suffisant pour un travail pratique de chat.

Opération	Paramètres	Réponse attendue	Rôle
subscribe	pseudo	OK ou ERROR	Inscrire un utilisateur.
postMessage	pseudo, message	OK	Ajouter un message à l'historique.
getMessages	lastIndex	tableau de chaînes	Retourner les nouveaux messages.
unsubscribe	pseudo	OK	Retirer l'utilisateur de la salle.

4.5 Commandes et résultat attendu

Commandes de test de la version XML-RPC.

```
cd C:\Users\DELL\Downloads\i-fall\TP0-XMLRPC
ant compile
ant run-server

# dans un autre terminal
ant run-client
```

Le serveur doit écouter sur `http://localhost:8080/xmlrpc`. Lorsque deux clients sont lancés, les messages apparaissent après interrogation régulière du serveur. Une légère latence peut exister, car la réception dépend de la fréquence de polling.

La limite principale est cette interrogation répétée. Si l'intervalle est trop long, les messages arrivent avec retard ; s'il est trop court, le serveur reçoit beaucoup de requêtes. Le compromis choisi est acceptable pour un travail pratique, car il met en évidence la différence entre callback et polling.

Chapitre 5 : Application ChatUser avec SOAP

Enveloppe XML, service HTTP et contrat WSDL

5.1 Contexte du travail pratique

SOAP est un protocole de services web basé sur XML. Il impose une structure de message plus formelle que XML-RPC : une enveloppe, un corps et éventuellement un en-tête. Dans le projet, le service est publié sur `/chat` et expose aussi un WSDL simplifié via `/chat?wsdl`.

5.2 Objectif du travail pratique

L'objectif est de comprendre comment un service web contractuel traite des opérations distantes. Par rapport à XML-RPC, SOAP rend l'échange plus formel : les opérations sont placées dans une enveloppe XML et le service peut fournir une description WSDL.

La réponse proposée consiste à garder la logique métier `ChatRoomImpl`, mais à l'envelopper dans une couche SOAP. Ainsi, le cœur de l'application reste comparable aux autres versions, tandis que la couche réseau illustre le traitement XML propre à SOAP.

5.3 Architecture du service

La version SOAP utilise `HttpServer` pour recevoir des requêtes HTTP. Une requête `GET` avec la query `wsdl` retourne le contrat du service. Une requête `POST` contient une enveloppe SOAP ; le serveur parse le XML avec `SoapUtil`, identifie l'opération demandée, puis appelle la méthode correspondante dans `ChatRoomImpl`.

La figure 5.1 présente traitement soap avec enveloppe xml et wsdl.

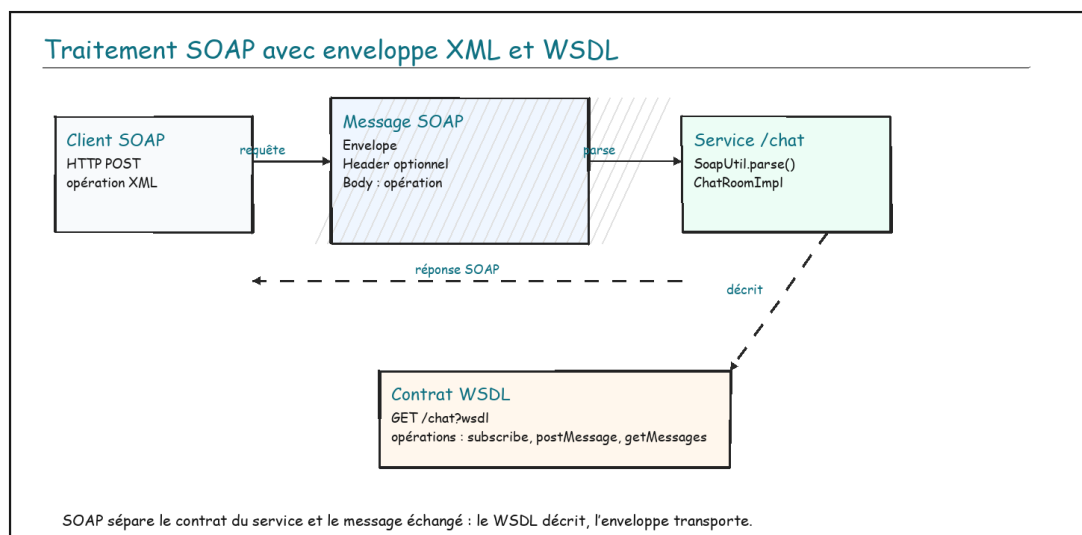


Figure 5.1 : Traitement SOAP avec enveloppe XML et WSDL

5.4 Analyse du traitement

Le code montre bien la séparation entre transport, parsing et logique métier. `ChatServer` reçoit la requête, `SoapUtil` extrait l'opération, puis `ChatRoomImpl` applique la règle fonctionnelle : inscription, désinscription, ajout d'un message ou récupération des messages.

Extrait ciblé : distinction entre WSDL et traitement d'une opération SOAP.

```
if ("GET".equals(exchange.getRequestMethod())
    && "wsdl".equals(exchange.getRequestURI().getQuery())) {
    send(exchange, 200, "text/xml; charset=UTF-8", wsdl());
    return;
}
```

```
Document document = SoapUtil.parse(request);
Element operation = SoapUtil.getSoapOperation(document);
```

Le WSDL joue le rôle de description du service : il indique le nom du service, l'adresse, les opérations disponibles et le binding SOAP. Même si le WSDL du projet est volontairement compact, il montre l'idée fondamentale : un service SOAP est censé être consommable à partir d'un contrat.

Composant	Responsabilité	Observation
-----------	----------------	-------------

Composant	Responsabilité	Observation
HttpServer	Recevoir les requêtes HTTP	Point d'entrée technique du service.
SoapUtil	Construire et parser les enveloppes XML	Isole la manipulation XML du serveur.
ChatRoomImpl	Appliquer la logique de chat	Reste proche des autres versions.
WSDL	Décrire le service	Aide les clients à connaître les opérations disponibles.

5.5 Commandes et résultat attendu

Commandes de lancement et vérification du WSDL SOAP.

```
cd C:\Users\DELL\Downloads\i-fall\TP0-SOAP
ant compile
ant run-server

# accès au contrat
http://localhost:8081/chat?wsdl
```

La page WSDL doit être accessible dans un navigateur. Les clients doivent ensuite pouvoir envoyer des messages et récupérer les nouveaux messages à travers des enveloppes SOAP. Cette version est plus verbeuse que REST, mais plus contractuelle.

La limite de SOAP dans ce projet est la lourdeur des messages XML. Pour un petit chat, REST est plus léger. Mais SOAP reste intéressant pédagogiquement, car il montre la logique des services fortement structurés, encore présents dans certains systèmes d'information.

Chapitre 6 : Application ChatUser avec REST

Endpoints HTTP, JSON et ressources applicatives

6.1 Contexte du travail pratique

La version REST traduit le fonctionnement de ChatUser en endpoints HTTP. Les opérations ne sont plus exposées comme des objets distants ou comme des procédures XML, mais comme des points d'accès web : inscription, envoi de message, consultation des nouveaux messages et désinscription.

6.2 Objectif du travail pratique

L'objectif est d'exprimer l'application sous forme d'API web simple. Le client n'a plus besoin d'une bibliothèque RPC particulière : il envoie des requêtes HTTP et reçoit du JSON. Ce modèle facilite les tests avec des outils standards et prépare à la conception d'API modernes.

La réponse proposée définit quatre endpoints, chacun associé à une responsabilité précise. Le serveur vérifie la méthode HTTP, lit les données envoyées et répond avec un statut JSON. Le comportement métier reste celui du chat, mais l'interface devient web.

6.3 Endpoints disponibles

Le serveur REST écoute sur le port 8082. Les échanges sont en JSON, ce qui rend les requêtes plus légères et lisibles que les enveloppes SOAP. Le client Swing peut utiliser ces endpoints, mais ils peuvent aussi être testés avec un navigateur, curl ou Postman.

La figure 6.1 présente architecture rest avec endpoints http et json.

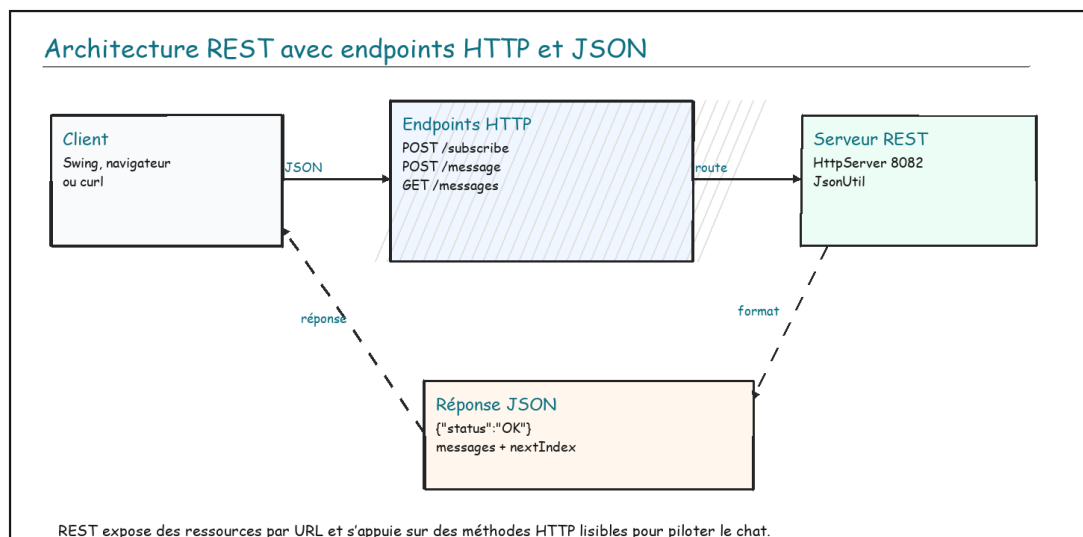


Figure 6.1 : Architecture REST avec endpoints HTTP et JSON

Endpoint	Méthode	Corps / paramètres	Rôle
/chat/subscribe	POST	{"pseudo": "Alice"}	Inscrire un utilisateur.
/chat/message	POST	{"pseudo": "Alice", "message": "Bonjour"}	Publier un message.
/chat/messages	GET	lastIndex=0	Lire les nouveaux messages.
/chat/unsubscribe	POST	{"pseudo": "Alice"}	Quitter la salle.

6.4 Analyse du code

La classe `ChatServer` crée un contexte HTTP pour chaque endpoint. Les méthodes `handleSubscribe`, `handleMessage` ou `handleMessages` vérifient la méthode HTTP, lisent le JSON, appellent la logique métier puis renvoient une réponse JSON avec un statut explicite.

Extrait ciblé : publication des endpoints REST et réponse JSON.

```
server.createContext("/chat/subscribe", ChatServer::handleSubscribe);
server.createContext("/chat/message", ChatServer::handleMessage);
server.createContext("/chat/messages", ChatServer::handleMessages);

exchange.getResponseHeaders();
```

```
.set("Content-Type", "application/json; charset=UTF-8");
```

Choix pédagogique

Le projet utilise un `JsonUtil` simple au lieu d'une grosse bibliothèque externe. Cela rend le mécanisme visible : lecture du corps, extraction des champs et construction manuelle de la réponse.

6.5 Commandes et résultat attendu

Commandes et exemple de requête pour la version REST.

```
cd C:\Users\DELL\Downloads\i-fall\TP0-REST
ant compile
ant run-server

# exemple de test conceptuel
POST http://localhost:8082/chat/subscribe
{"pseudo":"Alice"}
```

Le serveur doit afficher la liste des endpoints disponibles. Les clients doivent pouvoir s'inscrire, publier des messages et récupérer les messages avec `lastIndex`. REST est la version la plus proche des API web modernes.

La limite de cette version est que le projet reste volontairement simple : il n'y a pas d'authentification, pas de persistance et pas de vraie gestion d'erreurs avancée. Pour un travail pratique, cette simplicité est utile car elle laisse apparaître clairement la relation entre URL, méthode HTTP, JSON et logique métier.

Chapitre 7 : Comparaison générale des architectures

Lire les différences techniques et pédagogiques

7.1 Vue comparative

Les cinq versions résolvent le même problème, mais elles ne le modélisent pas de la même manière. TCP expose l'échange brut, RMI privilégie l'objet distant, XML-RPC privilégie l'appel de procédure simple, SOAP formalise les échanges XML avec un contrat, et REST transforme les actions en ressources HTTP manipulées avec JSON.

La figure 7.1 présente comparaison des modèles de communication distribuée.

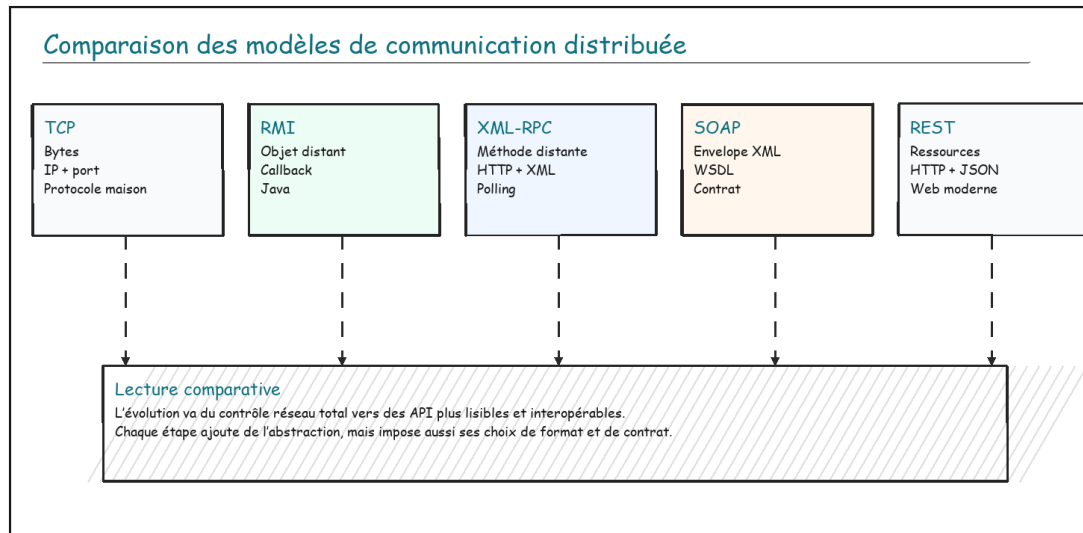


Figure 7.1 : Comparaison des modèles de communication distribuée

7.2 Tableau de synthèse

Critère	TCP	RMI	XML-RPC	SOAP	REST
Modèle	Flux brut	Objet distant	Procédure distante	Service contractuel	Ressource HTTP
Format	Texte/bytes	Sérialisation Java	XML	XML SOAP	JSON
Transport	TCP direct	RMI/JRMP	HTTP	HTTP	HTTP
Réception messages	Connexion persistante	Callback serveur	Polling	Polling	Polling
Interopérabilité	Possible mais manuelle	Faible hors Java	Bonne	Bonne mais verbeuse	Très bonne
Lisibilité réseau	Très visible	Peu lisible	Lisible mais XML	Très verbeux	Lisible et compact
Usage typique	Chat/jeu bas niveau	Système Java homogène	RPC léger	SI formel/contrat	API web moderne

7.3 Analyse

La différence la plus structurante concerne le sens de communication. RMI permet au serveur d'appeler directement les clients grâce à `ChatUser`. Les autres versions principales ne disposent pas de ce callback ; le client doit donc interroger le serveur pour récupérer les nouveaux messages. Cette distinction explique une grande partie des différences de code.

Le deuxième axe concerne le format. TCP rend le protocole applicatif entièrement manuel. SOAP et XML-RPC utilisent XML, ce qui facilite la représentation structurée mais augmente la verbosité. REST utilise JSON, plus compact et plus habituel dans les API modernes. RMI, lui, masque le format réseau au développeur Java.

Synthèse pédagogique

TCP montre la base réseau brute, RMI montre l'appel objet distant, XML-RPC montre le RPC simple, SOAP montre le service web contractuel, REST montre l'API web moderne. Le projet donne donc une comparaison complète, du plus bas niveau vers le modèle web actuel.

Chapitre 8 : Guide d'exécution et de validation

Compiler, lancer, capturer et contrôler les résultats

8.1 Préparation de l'environnement

Les projets sont des applications Java organisées avec Apache Ant. Avant de tester, il faut vérifier que Java et Ant sont disponibles dans le terminal. Ensuite, chaque version se lance selon le même principe : compilation, démarrage du serveur, lancement d'un ou plusieurs clients.

Contrôle minimal de l'environnement Java et Ant.

```
java -version
ant -version
```

La figure 8.1 présente chaîne de validation commune des applications chatuser.

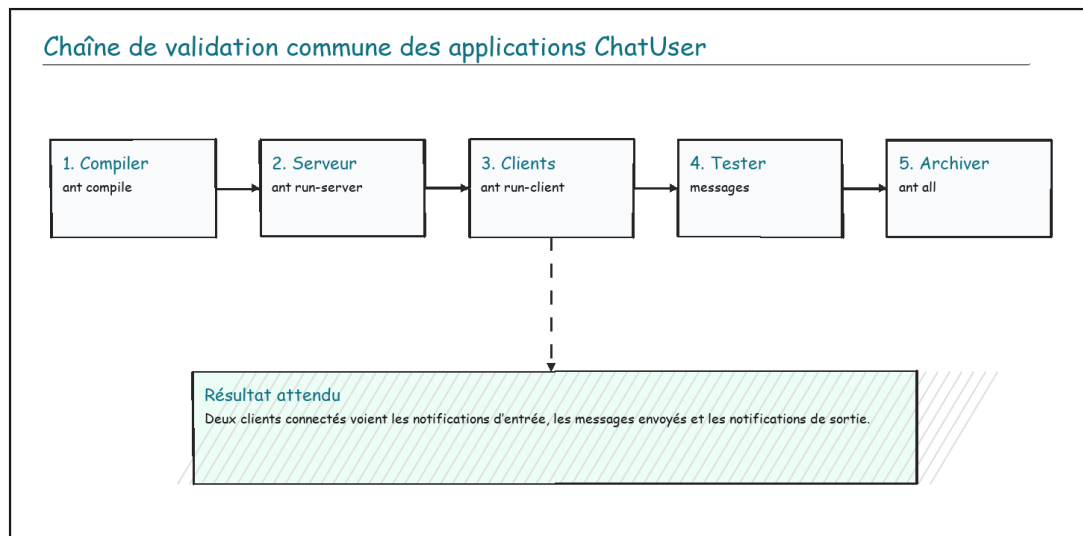


Figure 8.1 : Chaîne de validation commune des applications ChatUser

8.2 Scénario de test commun

Le scénario de validation doit rester identique pour toutes les versions principales. On lance un serveur, puis deux clients. Le premier client utilise le pseudo Alice, le second utilise Bob. Alice envoie un message, Bob doit le recevoir ; Bob répond, Alice doit le recevoir à son tour.

- Le serveur démarre sans erreur sur le port prévu.
- Alice peut rejoindre la salle de discussion.
- Bob peut rejoindre la même salle.
- Les notifications d'arrivée sont visibles.
- Un message envoyé par Alice apparaît chez Bob.
- Un message envoyé par Bob apparaît chez Alice.
- La sortie d'un client ne bloque pas le serveur.

8.3 Commandes par version

Version	Dossier	Port	Commandes principales
TCP	TP0-TCP	8083	ant compile ; ant run-server ; ant run-client
RMI	TP0	1099	ant compile ; ant run-server ; ant run-client -Dpseudo=Alice
XML-RPC	TP0-XMLRPC	8080	ant compile ; ant run-server ; ant run-client
SOAP	TP0-SOAP	8081	ant compile ; ant run-server ; ant run-client
REST	TP0-REST	8082	ant compile ; ant run-server ; ant run-client

8.4 Captures d'exécution à insérer

Les captures ci-dessous documentent une exécution réelle de la version REST de ChatUser. Elles complètent les schémas en montrant le serveur, les clients, l'échange de messages et la sortie d'un utilisateur.

Capture 8.1 : serveur REST démarré

```

PS C:\Users\DELL\Downloads\i-fall> cd .\TP0-REST\
PS C:\Users\DELL\Downloads\i-fall\TP0-REST> ant run-server
Buildfile: build.xml

init:

compile:
[javac] Compiling 5 source files to C:\Users\DELL\Downloads\i-fall\TP0-REST\bin

run-server:
[java] Serveur REST démarre sur http://localhost:8082
[java] Endpoints disponibles :
[java] POST /chat/subscribe {"pseudo":"Alice"}
[java] POST /chat/message {"pseudo":"Alice","message":"Bonjour"}
[java] GET /chat/messages?lastIndex=0
[java] POST /chat/unsubscribe {"pseudo":"Alice"}
[java] *** salif a rejoint la salle de discussion ***
  
```

Cette capture montre le lancement du serveur avec `ant run-server`. Le terminal affiche le port `8082`, les endpoints REST disponibles et les premières notifications d'arrivée dans la salle.

Capture 8.2 : deux clients connectés

```

PS C:\Users\DELL\Downloads\i-fall\TP0-REST> ant run-server
Buildfile: build.xml

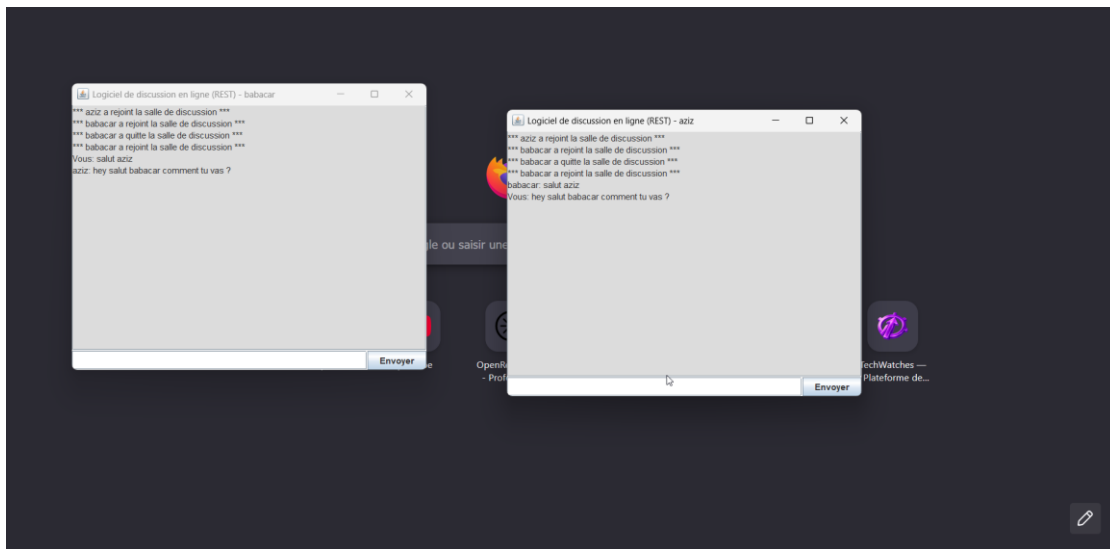
init:

compile:
[javac] Compiling 5 source files to C:\Users\DELL\Downloads\i-fall\TP0-REST\bin

run-server:
[java] Serveur REST démarre sur http://localhost:8082
[java] Endpoints disponibles :
[java] POST /chat/subscribe {"pseudo":"Alice"}
[java] POST /chat/message {"pseudo":"Alice","message":"Bonjour"}
[java] GET /chat/messages?lastIndex=0
[java] POST /chat/unsubscribe {"pseudo":"Alice"}
[java] *** aziz a rejoint la salle de discussion ***
[java] *** babacar a rejoint la salle de discussion ***
  
```

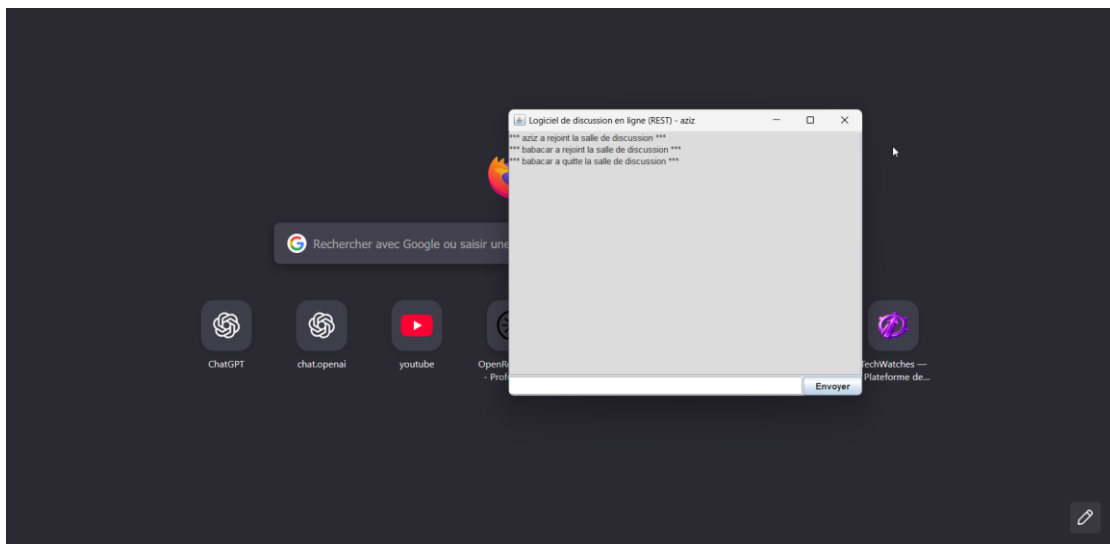
Cette capture confirme que deux utilisateurs distincts rejoignent la salle de discussion. Le serveur reçoit les événements et les affiche dans le terminal.

Capture 8.3 : message envoyé et reçu



Cette capture montre deux fenêtres clientes. Un message envoyé par un utilisateur apparaît dans l'autre fenêtre, ce qui valide la circulation des messages entre clients via le serveur REST.

Capture 8.4 : sortie de l'utilisateur Babacar



Cette capture illustre la notification de sortie d'un utilisateur. Elle permet de vérifier que la désinscription est prise en compte sans bloquer l'application.

8.5 Points de contrôle

La validation ne doit pas seulement consister à voir que le programme se lance. Il faut vérifier le comportement attendu : unicité des pseudos, réception des messages, absence de blocage serveur, gestion correcte des clients déconnectés et cohérence entre les messages affichés par les différents clients.

Bon réflexe de test

Toujours garder le terminal serveur visible pendant les tests : il indique les connexions, les messages reçus et les erreurs éventuelles.

Annexes techniques

Automatisation Ant, génération PDF FOP et références

Annexe A.1 : Projet Ant

Apache Ant automatise les tâches répétitives d'un projet Java : création des répertoires, compilation, génération de documentation, création d'une archive JAR et nettoyage. Le dossier `ProjetAnt` illustre cette logique avec un programme simple `Bonjour.java`.

La figure A.1 présente pipeline apache ant du projet.

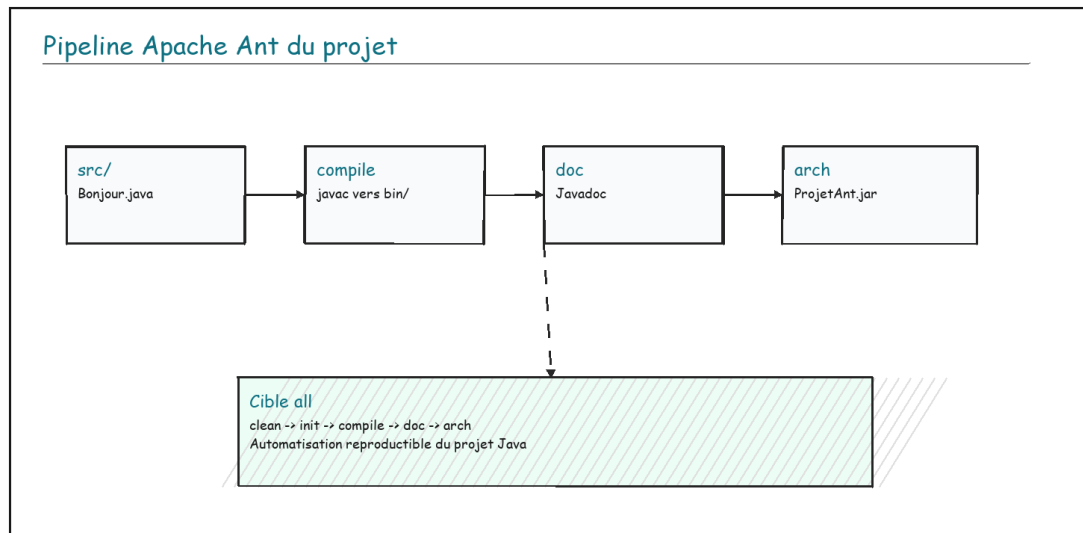


Figure A.1 : Pipeline Apache Ant du projet

Cible Ant	Rôle	Résultat attendu
init	Créer les répertoires nécessaires	`bin`, `doc` et `archive` sont disponibles.
compile	Compiler les sources Java	Les fichiers `.class` sont générés dans `bin`.
doc	Produire la documentation Javadoc	La documentation HTML est générée dans `doc`.
arch	Créer une archive exécutable	Un fichier JAR est produit dans `archive`.
clean	Nettoyer les sorties générées	Le dossier `bin` est supprimé pour repartir proprement.
run	Exécuter le programme	La classe `Bonjour` est lancée via Ant.
all	Enchaîner le build complet	Nettoyage, compilation, documentation et archive sont réalisés.

Extrait ciblé : cible Ant globale du projet.

```

<target name="all" depends="clean,init,compile,doc,arch">
  <echo message="Build complet terminé avec succès !"/>
</target>

```

L'intérêt d'Ant est la reproductibilité : au lieu d'expliquer une longue série de commandes, le projet documente les actions dans `build.xml`. Un autre étudiant peut relancer les mêmes étapes avec les mêmes noms de cibles.

Dans le cadre du projet i-fall, cette annexe explique aussi pourquoi chaque dossier ChatUser contient un `build.xml`. Le rapport met donc en relation les commandes utilisées dans les chapitres et le mécanisme d'automatisation qui les rend reproductibles.

Annexe A.2 : Génération PDF avec Apache FOP

Apache FOP transforme des documents XSL-FO en sorties imprimables comme PDF. Dans le dossier `fop`, le fichier `equipe.xml` contient les données, tandis que `statistiques-fo.xml` décrit la mise en forme. Le dossier `projet` contient aussi un exemple Java `Fo2Pdf.java` qui pilote FOP côté code.

La figure A.2 présente pipeline apache fop de génération pdf.

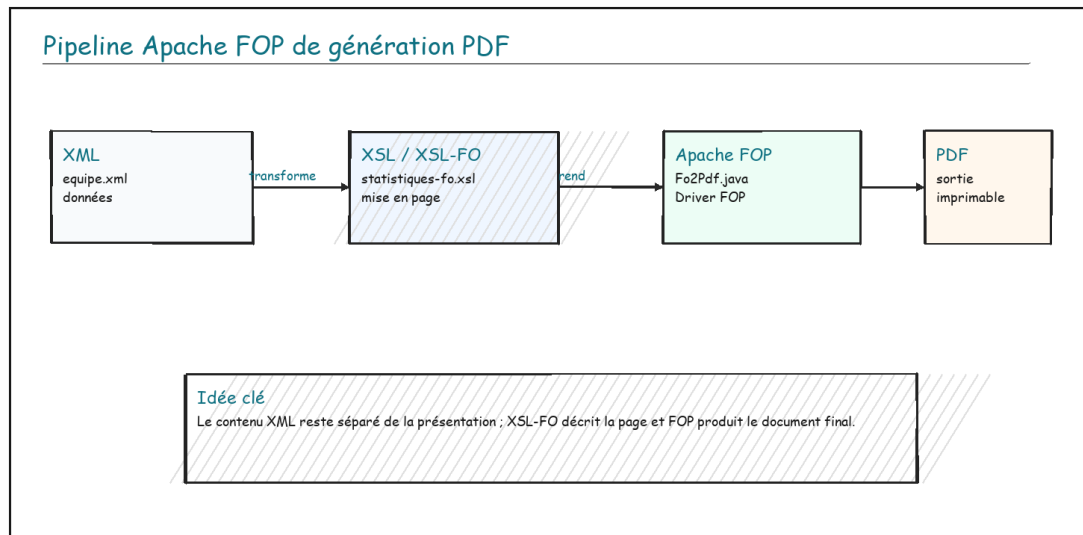


Figure A.2 : Pipeline Apache FOP de génération PDF

Fichier / dossier	Rôle	Explication
equipe.xml	Source de données	Contient les équipes, victoires, défaites et classements.
statistiques.xml	Transformation HTML/XML	Présente une transformation lisible côté navigateur ou XML.
statistiques-fo.xml	Transformation XSL-FO	Décrit la mise en page destinée au rendu PDF.
test.fo	Document XSL-FO direct	Sert de fichier d'entrée simple pour tester FOP.
Fo2Pdf.java	Programme Java	Configure le driver FOP et produit le PDF à partir du fichier FO.
out/test.pdf	Sortie générée	Montre le résultat final après conversion.

Extrait ciblé : conversion XSL-FO vers PDF avec Apache FOP.

```

Driver driver = new Driver();
driver.setRenderer(Driver.RENDER PDF);
driver.setInputSource(new InputSource(in));
driver.setOutputStream(out);
driver.run();
  
```

La séparation entre XML et XSL-FO est importante : les données restent propres et indépendantes, tandis que la feuille de transformation décide de la présentation finale. C'est une logique proche des architectures qui séparent modèle, traitement et rendu.

Cette annexe complète les travaux sur les services distribués en montrant une autre forme de chaîne technique : une donnée structurée est transformée en document final. Le principe général reste similaire à une architecture propre : séparer l'information, la transformation et le format de sortie.

Annexe A.3 : Références consultées

Référence	Lien	Utilisation dans le rapport
Oracle Java RMI API Guide	https://docs.oracle.com/en/java/javase/11/rmi/index.html	Définition et logique de Java RMI.
Apache XML-RPC	https://ws.apache.org/xmlrpc/	Description du protocole XML-RPC et de l'implémentation Java.
W3C SOAP 1.1	https://www.w3.org/TR/SOAP/	Structure des messages SOAP.
W3C WSDL	https://www.w3.org/TR/wSDL.html	Rôle du contrat WSDL.
MDN REST	https://developer.mozilla.org/en-US/docs/Glossary/REST	Rappel sur REST comme style architectural.
Apache Ant Manual	https://ant.apache.org/manual/	Cibles et automatisation de build.
Apache FOP Quick Start	https://xmlgraphics.apache.org/fop/quickstartguide.html	Principe de génération PDF avec FOP.

Conclusion générale

Le projet i-fall permet de comprendre que la communication distribuée ne dépend pas seulement du code métier, mais aussi du modèle d'échange choisi. TCP montre la base réseau brute, RMI donne une vision

objet, XML-RPC simplifie l'appel distant, SOAP formalise les contrats XML et REST propose une approche web lisible et interopérable. Les annexes Ant et FOP complètent le travail en montrant l'automatisation de build et la transformation documentaire.

Bilan

Le livrable final met donc en relation le code, les architectures, les commandes de test et les choix techniques, afin que le lecteur comprenne le projet au lieu de seulement voir une liste de fichiers.

Code source du projet : <https://github.com/salifbiaye/projet-tp-0>